BLOCKCHAIN-BASED DIGITAL TICKET AUTHENTICITY & RESALE VERIFICATION SYSTEM

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Use Case Report

submitted by

ADDEPALLI PAVITHRA LAKSHMI

24505A0501

Under the guidance of

Mr. A. Prashant, Asst. Prof.



Department of Computer Science and Engineering

Prasad V Potluri Siddhartha Institute of Technology

(Permanently affiliated to JNTU-Kakinada, Approved by AICTE)

(An NBA & NAAC accredited and ISO 9001:2015 certified institute)

Kanuru, Vijayawada-520 007

2026-27

Prasad V Potluri Siddhartha Institute of Technology

(Permanently affiliated to JNTU-Kakinada, Approved by AICTE)

(An NBA & NAAC accredited and ISO 9001:2015 certified institute)

Kanuru, Vijayawada-520 007

CERTIFICATE

This is to certify that the Use Case report entitled **"Blockchain-Based Digital Ticket Authenticity & Resale Verification System"** that is being submitted by **Addepalli Pavithra Lakshmi(24505A0501)**, as part of Assignment-2 for the **Blockchain Technology(23CS4701B)** course in **4-1** during the academic year **2026-27**.

Course Coordinator
Mr. A. Prashant
Assistant Professor,
Department of CSE,
PVPSIT, Vijayawada

Head of the Department
Dr. A. Jayalakshmi,
Professor and Head,
Department of CSE,
PVPSIT, Vijayawada

MARKS
ASSIGNMENT-2: _____ /5

INDEX

S. No.	Chapter	Page No.
1	Introduction	01
2	Background	02
3	Blockchain Basics	04
4	Use Case Overview	06
5	Implementation	09
6	Benefits	15
7	Challenges	17
8	Conclusion	18
9	SDG’s Addressed	19
10	References	20
11	Appendix A	22

1. INTRODUCTION

1.1 Overview of Blockchain Technology

Blockchain technology is a decentralized and distributed ledger system that records transactions across multiple nodes, ensuring transparency, security, and immutability. Unlike traditional databases, blockchain operates on a peer-to-peer network, eliminating the need for central authorities. Each block in the chain contains a cryptographic hash of the previous block, making data tampering virtually impossible. Originally designed for cryptocurrency transactions, blockchain has now expanded into various domains, including finance, supply chain, healthcare, and digital warranty management. [2]

1.2 Relevance of Blockchain in Digital Ticketing Systems

Traditional digital ticketing systems generally depend on centralized platforms to issue tickets, store ownership information, and process transfers. Although online ticketing has simplified ticket purchasing, problems such as counterfeit tickets, duplicate ticket distribution, unauthorized resale, and limited transparency in secondary markets continue to present challenges.

A blockchain-based digital ticketing system can address some of these problems by maintaining verifiable ticket records and enforcing predefined rules through smart contracts. Each ticket can be assigned a unique identifier, allowing the application to distinguish individual tickets and check their current status. Ownership transfers can be recorded as blockchain transactions, providing a traceable history of changes in ownership.

Smart contracts can also enforce resale conditions, such as restricting transfers to authorized users or applying a maximum resale price. These conditions are evaluated by the contract when a transaction is submitted. This reduces dependence on manual enforcement, provided that the rules are correctly implemented and cannot be bypassed through alternative transaction paths.

Another important application is ticket authenticity verification. Before accepting a ticket, a verification mechanism can check whether the ticket exists, whether it has been transferred to the current holder, and whether it has already been used or invalidated, if the system maintains such a status. This can reduce the risk of duplicate use and unauthorized ticket circulation.

Furthermore, blockchain records can improve transparency between event organizers, ticket holders, and authorized resale participants. By providing a consistent record of supported ticket transactions, the system can help resolve ownership disputes and improve confidence in secondary ticket markets.

2. BACKGROUND

Traditional ticketing systems rely on centralized platforms to issue tickets, maintain ownership records, and manage resale transactions. Despite the convenience of online booking, these systems face several challenges related to authenticity, transparency, and unauthorized resale. Below are some key issues with the existing systems:

2.1. Counterfeit and Duplicate Tickets

Digital tickets can be copied and distributed to multiple buyers. Without reliable verification, several people may attempt to use the same ticket, resulting in financial losses and entry disputes.

2.2. Unauthorized Resale and Ticket Scalping

Tickets purchased at their original price are sometimes resold at significantly higher prices. Traditional platforms may have limited control over transactions conducted through unauthorized secondary markets.

2.3. Lack of Ownership Transparency

Buyers may find it difficult to confirm whether a seller is the legitimate ticket owner or whether the ticket has already been transferred. This uncertainty reduces trust in peer-to-peer ticket transactions.

2.4. Limited Traceability of Ticket Transfers

When tickets change hands multiple times, maintaining an accurate ownership history can be challenging. Insufficient transaction records can make it difficult to investigate disputes and identify unauthorized transfers.

2.5. Dependence on Centralized Platforms

Traditional ticketing systems depend on centralized databases for ticket information and validation. Problems such as service outages, unauthorized database modifications, or compromised accounts can affect ticket management and verification.

2.6. Challenges in Ticket Authenticity Verification

Buyers and event organizers need a reliable way to determine whether a ticket is genuine, valid, and associated with its current holder. Manual verification and poorly integrated systems may not provide sufficient protection against fraudulent tickets.

2.7. Need for a Blockchain-Based Ticketing System

These challenges highlight the need for a secure and transparent ticket management mechanism. Blockchain technology can provide verifiable transaction records, while Solidity smart contracts can enforce ticket ownership and resale rules. Such a system can improve traceability and reduce opportunities for fraud, provided that the contracts and ticket validation mechanisms are implemented correctly.

3. BLOCKCHAIN BASICS

Blockchain technology is a distributed ledger system that ensures secure, transparent, and tamper-proof transactions. Unlike traditional systems that rely on intermediaries, blockchain enables trustless peer-to-peer interactions, reducing the need for third parties in financial transactions, supply chain management, and digital warranty verification. Its decentralized nature enhances security and transparency, making it an ideal solution for many industries. Below are the key concepts that define blockchain technology:

3.1. Decentralization

Traditional databases are managed by centralized authorities, such as banks or corporations, which creates a single point of failure and makes them vulnerable to hacks or data manipulation. Blockchain, on the other hand, operates on a decentralized network, where multiple independent nodes validate and store data. This ensures that no single entity has full control over the system, making it resistant to censorship and fraud. Additionally, decentralized structures provide greater security, as compromising one node does not affect the integrity of the entire network. Since all transactions are recorded and verified across multiple nodes, blockchain also enhances transparency, allowing participants to independently verify transactions. [2]

3.2. Immutability

One of blockchain's defining characteristics is immutability, meaning once a transaction is recorded, it cannot be altered or deleted. [2] This is achieved through cryptographic hashing, where each block is linked to the previous one, forming a secure chain that prevents unauthorized modifications. The integrity of the blockchain is maintained through consensus mechanisms, ensuring that all network participants agree before new data is added. Since tampering with a single block would require modifying all subsequent blocks—an operation that demands enormous computational power—fraud and unauthorized changes become nearly impossible. This feature makes blockchain particularly useful for audit trails, financial transactions, and warranty tracking.

3.3. Transparency

Blockchain transactions are publicly verifiable, meaning anyone with access to the network can audit the transaction history. This transparency is ensured by distributed ledger technology (DLT), where every participant has access to an identical copy of the data, eliminating the risk of hidden alterations. Public blockchains, such as Bitcoin and Ethereum, offer complete transparency, fostering trust among users. This openness reduces the possibility of corruption and fraudulent activities, as any attempt to manipulate records would be instantly detected by the network. [2]

3.4. Smart Contracts

Smart contracts are self-executing contracts with predefined conditions embedded in the blockchain. Once the specified conditions are met, the contract automatically executes the agreed-upon actions, eliminating the need for manual processing. This automation ensures trustless execution, reducing fraud and disputes between parties. By removing intermediaries, smart contracts also lower operational costs and increase efficiency. In the context of a digital warranty system, smart contracts can automatically transfer warranty ownership, validate claims, and process warranty expirations without human intervention.[2] [4]

3.5. Consensus Mechanisms

Blockchain networks rely on consensus mechanisms to validate transactions and maintain system integrity. The most widely used mechanism is Proof of Work (PoW), where miners solve complex mathematical puzzles to confirm transactions, as seen in Bitcoin. Another approach, Proof of Stake (PoS), selects validators based on the number of tokens they hold, which is used in Ethereum 2.0. A more efficient variation, Delegated Proof of Stake (DPoS), involves electing a smaller group of nodes to verify transactions, improving scalability and reducing energy consumption. These consensus models prevent malicious actors from manipulating the system and ensure that only valid transactions are recorded on the blockchain. [5]

3.6. Cryptographic Security

Blockchain security relies on advanced cryptographic techniques to protect user data and transactions. Each transaction is authenticated using public and private keys, ensuring that only authorized individuals can initiate transactions. Hash functions further enhance security by converting data into fixed-length unique codes, making it nearly impossible to reverse-engineer original information. Additionally, blockchain uses encryption to protect sensitive data, ensuring that unauthorized users cannot access confidential records. This high level of security makes blockchain an ideal technology for digital warranties, financial transactions, and identity management.

By combining these key features—decentralization, immutability, transparency, smart contracts, and cryptographic security—blockchain provides a robust and efficient foundation for various industries, including digital warranty management. It ensures secure, automated, and verifiable transactions, reducing fraud and increasing trust in warranty claims and ownership transfers.

These fundamental blockchain principles make it secure, transparent, and efficient, enabling applications in finance, healthcare, supply chains, and digital warranties.

4. USE CASE OVERVIEW

The Blockchain-Based Digital Ticket Authenticity and Resale Verification System aims to improve digital ticket management using blockchain technology and Solidity smart contracts. The system provides a mechanism for issuing unique tickets, verifying ownership, transferring tickets, and enforcing resale rules. By maintaining verifiable transaction records, it seeks to reduce counterfeit tickets and unauthorized resale..

4.1. Objectives

The primary objective of the proposed system is to provide a reliable method for managing digital event tickets using blockchain technology.
The key objectives are:

- **Ticket authenticity**: Assign a unique identifier to each ticket and support verification of its validity.
- **Ownership verification**: Ensure that only the current authorized owner can initiate a ticket transfer.
- **Controlled resale**: Enforce predefined resale conditions, such as maximum resale prices, through smart contracts.
- **Transparent transactions**: Maintain a verifiable record of ticket issuance and ownership transfers.
- **Duplicate-use prevention**: Track ticket status and support marking tickets as used or invalidated during event entry.
- **Reduced fraud**: Minimize opportunities for counterfeit ticket distribution and unauthorized transfers.

4.2. Scope of the System

The proposed system focuses on blockchain-based ticket issuance, verification, ownership transfer, and resale management. Its primary components include :

1. **Event Organizers: Create events and issue digital tickets to buyers.** [3]
2. **Ticket Buyers:** Purchase tickets, view ticket details, verify authenticity, and transfer or resell eligible tickets**.**
3. **Resale Participants:** Purchase tickets through an authorized resale mechanism that follows predefined contract rules.
4. **Blockchain Network**: Maintains ticket records and executes smart contracts governing issuance, transfers, and resale.
5. **Entry Verification Staff**: Check ticket validity and usage status before allowing event entry.

4.3. System Architecture

The architecture of the proposed Blockchain-Based Digital Ticket Authenticity and Resale Verification System consists of the following layers

4.3.1. User Layer

This layer includes the participants who interact with the ticketing platform.

- **Event Organizers**: Manage event information and issue tickets.
- **Ticket Buyers**: View tickets and verify ownership.
- **Resale Participants**: Transfer or purchase tickets under the permitted resale rules.
- **Entry Verification Staff**: Validate tickets at the event venue.

4.3.2. Web Application Layer

The web application provides an interface between users and the blockchain network.

- **User Interface**: Enables users to access event information, manage tickets, and initiate permitted operations.
- **Wallet Integration**: Connects users' blockchain accounts to the application and enables transaction signing.
- **Ticket Management Module**: Coordinates ticket creation, ownership queries, transfers, and verification requests.
- **Resale Module**: Presents eligible tickets for resale and submits transactions subject to the smart contract's rules.

4.3.3. Blockchain Layer

This layer maintains ticket-related records and enforces the rules implemented in Solidity.

- **Ticket Smart Contract**: Manages unique ticket identifiers, ownership records, and ticket status.
- **Resale Smart Contract Logic**: Validates resale conditions, including applicable price restrictions, when implemented.
- **Transaction Records**: Maintain a verifiable history of supported ticket operations.
- **Ticket Verification**: Checks the ticket's recorded status and ownership before an authorized operation is accepted.

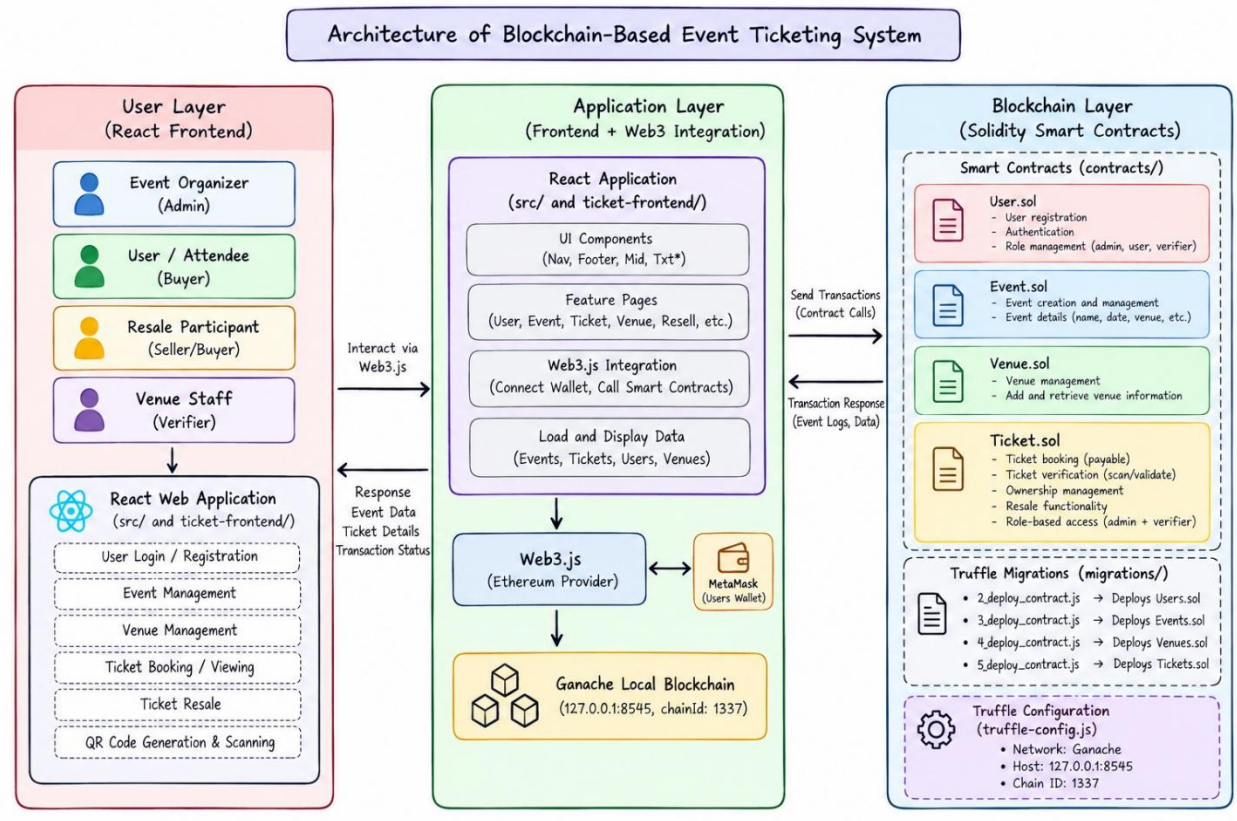


Figure 4.1: Architecture of Blockchain based Digital Warranty System

5.IMPLEMENTATION

5.1. Setting Up the Blockchain Environment

The development environment uses Solidity for smart-contract development, Truffle for deployment, Ganache as a local blockchain, and React with Web3.js for frontend interaction. The project configuration identifies the local blockchain endpoint as 127.0.0.1:8545 with chain ID 1337.

The four smart contracts are deployed independently using Truffle migration scripts. After deployment, the frontend must be configured with the deployed contract addresses and corresponding ABI definitions to interact with the contracts.

The major components of the environment are:

- **Solidity**: Implements ticket, user, event, and venue management logic.
- **Truffle**: Compiles and deploys the smart contracts.
- **Ganache**: Provides a local blockchain for development and testing.
- **React**: Provides the web interface for the different ticketing operations.
- **Web3.js**: Connects the frontend to the blockchain and invokes contract functions.
- **MetaMask**: Can be used to connect a wallet and authorize blockchain transactions, if configured in the application.

5.2. Defining Smart Contracts

5.2.1. User Registration and Management

The Users.sol contract manages user information using the UserLocked structure. It stores fields such as name, email, username, password, blockchain address, phone number, age, role, verification status, and identity-photo address.

A mapping associates each blockchain address with its user record, while an array stores registered user addresses. The createUserLocked() function creates a user record using the address of the transaction sender. Functions such as getUserLocked() and getAllUsersLocked() retrieve user information, while updateUserLocked() modifies selected fields.

The contract also provides functions to check account existence and verify a submitted password against the stored value. However, the supplied implementation stores passwords directly in blockchain state. Hashing a password during comparison does not make the stored password confidential; this design should be treated as a security limitation and improved before real-world use..

5.2.2. Creating and Managing Events

The Events.sol contract manages event information and ticket-pricing details. The EventLocked structure stores the event ID, owner address, venue ID, event name, event type, description, contact number, status, image address, event date, and timestamps.

The contract uses mappings to store event details, ticket-pricing information, and payment details. The createEventLocked() function creates an event record and associates it with the blockchain address of the caller. The createEvent2Locked() function stores ticket categories, normal and VIP prices, ticket quantities, and the minimum age associated with an event.

```
struct EventLocked {
    string eventId;
    address owner;
    string venueId;
    string eventName;
    string eventType;
    string description;
    uint eventPhoneNo;
    string status;
    string eventPhotoAddress;
    uint eventDate;
    uint256 dateCreated;
    uint256 dateModified;
}
```

5.2.3. Venue Management

The Venues.sol contract manages venue information and availability. Its VenueLocked structure contains the venue ID, owner address, venue name, venue type, location, description, contact number, maximum capacity, status, and timestamps.

The createVenueLocked() function registers a venue and records the caller as its owner. The contract also supports adding venue photographs, payment details, and services such as catering, parking, staffing, and audiovisual facilities.

The addBookedDate() function records a date associated with a venue, while checkIsDateBooked() checks whether a date is already recorded as booked. Other functions retrieve venue details, list venues, and retrieve the venues associated with the caller.

An important limitation is that several management functions, including venue status updates and deletion, do not consistently enforce ownership authorization. The date-booking logic also requires further testing to ensure that availability checks work correctly for all cases

```
//get all the venue ID's
    function getVenueIDList() public view returns(string[] memory) {
        return venueAccounts;
    }

    function getBookedId() public view returns(uint[] memory) {
        return _bookedId;
    }

    function getAllBookedDates() public view returns(BookedDates[] memory) {
        BookedDates[] memory tempDates = new BookedDates[](_bookedId.length);
        for (uint256 i=0; i<_bookedId.length; i++){
            tempDates[i]= bookedDate[i];
        }
        return tempDates;
    }

//check if the venue ID is available
    function checkVenueIDExistance(string memory tempVenueID)
    public view returns(bool){
        for (uint256 i=0; i<venueAccounts.length; i++){
            if
(keccak256(abi.encodePacked(tempVenueID))==keccak256(abi.encodePacked(venueA
ccounts[i])))
            return true;
        }
        return false;
    }
```

5.2.4. Viewing Warranties Owned by a Consumer

Consumers must have the ability to access and review the details of their warranty tokens. This functionality is achieved through a function that retrieves all warranties linked to the caller's blockchain address. The function loops through the stored warranty tokens and returns a structured list containing warranty details such as product information, issuance date, and expiration date. This ensures transparency and enables consumers to track their warranty coverage.

```
function viewWarranties() external view returns (Warranty[] memory) {
    uint256[] memory tokenIds = ownerToTokens[msg.sender];
    Warranty[] memory ownedWarranties = new Warranty[](tokenIds.length);

    for (uint256 i = 0; i < tokenIds.length; i++) {
        ownedWarranties[i] = warranties[tokenIds[i]];
    }

    return ownedWarranties;
}
```

5.2.5. Ticket Booking and Management

The Tickets.sol contract implements the core ticket-booking functionality. It maintains ticket records and event-level information, including maximum seats, booked seats, and ticket price.

The TicketLocked structure stores the ticket ID, ticket identifier, owner address, event ID, ticket level, scanned status, resale price, creation timestamp, and modification timestamp.

```
struct TicketLocked {
    string ticketId;
    string ticketIdentifier;
    address owner;
    string eventId;
    string ticketLevel;
    bool isScanned;
    uint resellPrice;
    uint256 dateCreated;
    uint256 dateModified;
}
```

5.2.6. Ticket Verification and Entry Validation

Ticket verification is handled by the scanTicketLocked() function in Tickets.sol. The contract defines an administrator and a designated verifier address. The onlyVerifierOrAdmin modifier restricts ticket scanning to either of these addresse. [5]

Before marking a ticket as scanned, the function checks whether the ticket exists and whether it has already been scanned.

After successful validation, the contract updates the ticket's scanned status and modification timestamp. A subsequent scanning attempt is rejected, helping prevent repeated use of the same ticket through this function.

The getTicketLocked() function retrieves the ticket record for a supplied ticket ID. The frontend can use the returned information to display ticket details and its scanning status.

```
function scanTicketLocked(
    string memory tempTicketID
) public onlyVerifierOrAdmin {
    require(
        tickets[tempTicketID].owner != address(0),
        "Ticket not exist"
    );
    require(
        !tickets[tempTicketID].isScanned,
        "Already scanned"
    );

    tickets[tempTicketID].isScanned = true;
    tickets[tempTicketID].dateModified = block.timestamp;
}
```

5.3. Deploying and Integrating with a Frontend

The React frontend provides the user interface for interacting with the ticketing system. The supplied repository structure includes pages for user registration and login, event creation and management, venue management, ticket booking, QR-code generation and scanning, and ticket resale.

Web3.js acts as the communication layer between the frontend and the blockchain. It enables the application to call read-only contract functions and submit transactions that modify blockchain state.

5.4. Workflow:

The workflow of the Blockchain-Based Digital Ticket Authenticity and Resale Verification System consists of the following stages:

1. **User registration**: A user creates a profile through the frontend, and the Users.sol contract stores the supplied user information.

2. **Venue registration**: Venue information is recorded through Venues.sol, including its name, type, location, and capacity.

3. **Event creation**: Event information and ticket-pricing details are recorded through Events.sol and, for booking capacity and price, through the relevant functions in Tickets.sol.

4. **Ticket booking**: The buyer submits a booking request and payment. The ticket contract validates event availability, checks payment, creates the ticket record, and updates the booked-seat count.

5. **Ticket retrieval**: The application retrieves the ticket record and displays the relevant details to the user.

6. **Entry verification**: An authorized administrator or verifier scans the ticket using the verification interface. If the ticket exists and has not been scanned, the contract marks it as scanned.

7. **Resale extension**: A future resale module can validate seller ownership and resale conditions before transferring a ticket to another buyer.

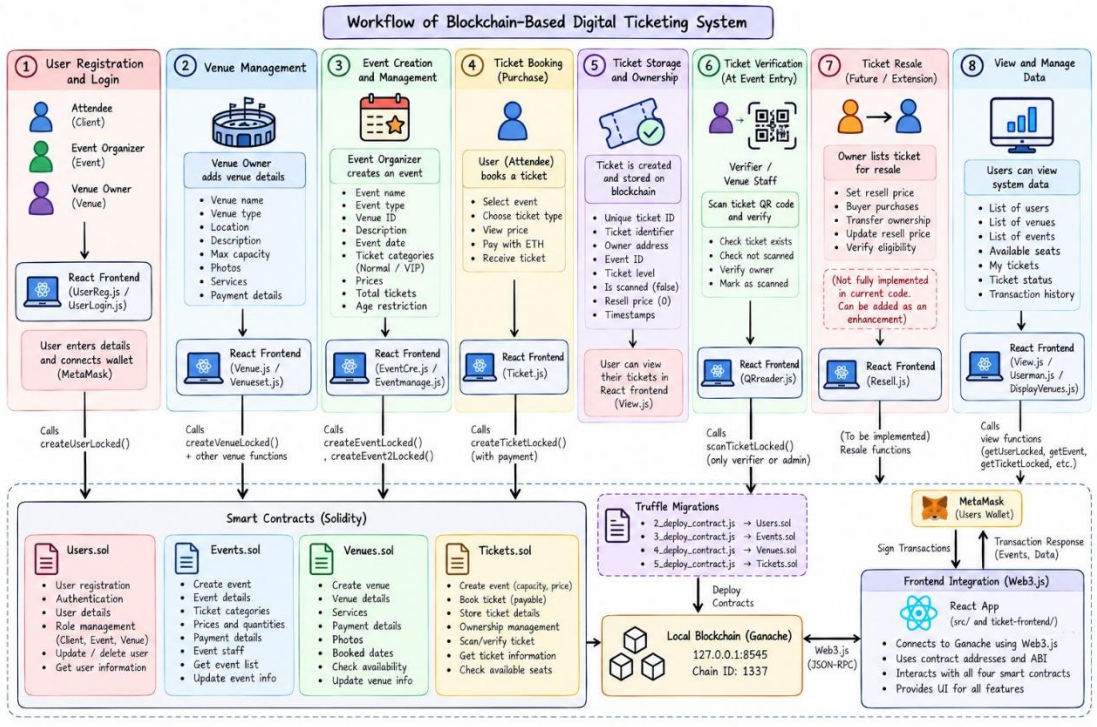


Figure 5.1: Workflow of Blockchain based Digital Warranty System

6. BENEFITS

The Blockchain-Based Digital Ticket Authenticity and Resale Verification System offers several benefits through the use of Solidity smart contracts and blockchain-based ticket records. Below are the key advantages:

6.1. Improved Ticket Authenticity

Each ticket is associated with a unique ticket ID and recorded in the smart contract. This allows the system to check whether a ticket exists and retrieve its ownership and scanning status, helping reduce the risk of counterfeit tickets. [1]

6.2. Secure Ticket Ownership Records

The ticket contract records the owner's blockchain address when a ticket is booked. This provides a verifiable ownership record and establishes a foundation for implementing secure ticket transfers. [2]

6.3. Prevention of Counterfeit and Duplicate Tickets

Each ticket is associated with a unique ticket ID. The createTicketLocked() function checks whether the ticket already exists before creating a new record. This reduces the risk of duplicate ticket IDs within the contract.

The scanTicketLocked() function also checks whether a ticket has already been scanned and rejects subsequent scanning attempts. Together, these mechanisms support ticket authenticity checks and help prevent repeated use through the implemented verification function.

6.4. Protection of Ticket Ownership Records

Traditionally, transferring product warranties between owners is complex and requires paperwork. With blockchain, warranty tokens are easily transferable between users. When a product is resold, the new owner automatically receives the warranty without requiring approval from the manufacturer, ensuring smooth resale transactions. [3]

6.5. Cost Reduction and Efficiency

Blockchain significantly reduces administrative costs associated with warranty management. Since all warranty records are stored on-chain, there is no need for manual verification, paperwork, or intermediaries, leading to faster claims processing and lower operational expenses for companies.

6.6. Security and Privacy Limitations

Although blockchain provides verifiable transaction records, it does not automatically guarantee the confidentiality of stored information. The supplied Users.sol contract stores passwords and personal details directly in contract state. Such information should not be stored in plaintext on a blockchain because blockchain data may be publicly accessible and difficult to remove.

A safer design would use secure off-chain authentication, password hashing, and restricted storage for personal information while recording only the minimum necessary information on-chain.

Overall, the system demonstrates basic security mechanisms for ticket booking and verification, but stronger access controls, secure handling of user data, and a complete resale implementation are necessary before real-world deployment.

6.7. Enhanced Security and Access Control

The Blockchain-Based Digital Ticket Authenticity and Resale Verification System uses Solidity smart contracts to manage ticket records and restrict selected operations to authorized blockchain addresses. In the Tickets.sol contract, the onlyAdmin modifier restricts event creation to the administrator, while onlyVerifierOrAdmin allows only the administrator or designated verifier to scan tickets.

These access-control mechanisms help prevent unauthorized ticket verification operations. However, access control is not consistently enforced across all four contracts, so additional authorization checks are required to secure event, venue, and user management.

7. CHALLENGES

Although the Blockchain-Based Digital Ticket Authenticity and Resale Verification System offers advantages in ticket management and verification, several challenges must be addressed to ensure its reliability, security, and practical usability.

7.1. Scalability Issues

Blockchain networks have limitations in transaction throughput and processing speed. During popular events, a large number of users may attempt to book tickets simultaneously, potentially increasing transaction delays and gas fees. A scalable blockchain network is necessary to handle high transaction volumes efficiently..

7.2. Security and Smart Contract Vulnerabilities

Smart contracts must be implemented carefully because coding errors can lead to unauthorized operations, incorrect payments, or inconsistent ticket records. In the current implementation, some event and venue management functions lack adequate ownership restrictions. Security reviews and comprehensive testing are required before deployment.

7.3. Lack of User Awareness and Adoption

Many businesses and consumers are not familiar with blockchain technology, which leads to resistance to adoption. Educating customers and warranty service providers on how to access and verify blockchain-based warranties remains a key challenge.

7.4. Dependence on Internet Connectivity

Since blockchain-based warranties rely on internet access for verification and transactions, users in areas with poor connectivity might face difficulties in accessing and managing their warranties.

7.5. Legal and Operational Considerations

Real-world deployment may require compliance with applicable data-protection laws, payment regulations, event-organizer policies, and consumer-protection requirements. Blockchain records alone do not guarantee admission to an event or establish the legal validity of every ticket transaction.

8. CONCLUSION

The Blockchain-Based Digital Ticket Authenticity and Resale Verification System demonstrates the application of blockchain technology and Solidity smart contracts to digital event ticket management. Traditional ticketing systems face challenges such as counterfeit tickets, duplicate usage, unauthorized resale, and limited ownership traceability. The proposed system addresses these challenges by maintaining ticket records on a blockchain and automating selected ticketing operations through smart contracts.

The implementation consists of four smart contracts: Users.sol, Events.sol, Venues.sol, and Tickets.sol. These contracts manage user information, event details, venue information, ticket booking, payment processing, and ticket verification. The React frontend, Web3.js integration, Truffle deployment framework, and Ganache local blockchain provide the application environment for interacting with the contracts.

The Tickets.sol contract supports ticket booking by checking event availability, validating payment, assigning ownership, and updating the number of booked seats. It also provides a verification mechanism that allows an authorized administrator or verifier to mark a ticket as scanned and prevents subsequent scans through the same function. These features demonstrate how smart contracts can support ticket management and reduce opportunities for repeated ticket usage.

However, the current implementation has limitations, including insufficient access controls in some contracts, insecure storage of personal information, and incomplete ticket resale functionality. These issues must be addressed before the system can be considered suitable for real-world deployment.

8.1. Future Outlook for Enhancements

Future improvements can include implementing a complete resale mechanism with price restrictions, secure payment processing, and ownership transfers. Stronger role-based access control can be introduced across all contracts, while user authentication and confidential information should be managed securely off-chain.

Additional enhancements may include integrating reliable QR-code validation, improving transaction efficiency, conducting smart-contract security audits, and deploying the application on an appropriate blockchain test network before considering production use.

In conclusion, the project provides a practical foundation for exploring blockchain-based ticket authenticity and verification. With further development and testing, it has the potential to improve transparency, ticket traceability, and trust among event organizers, ticket buyers, and authorized verification staff.

.

9. SDG's ADDRESSED

The Blockchain-Based Digital Ticket Authenticity and Resale Verification System aligns with United Nations Sustainable Development Goal 9 (SDG 9) by promoting technological innovation, transparent transactions, and reliable digital infrastructure.

SDG 9: Industry, Innovation, and Infrastructure

The project supports SDG 9 by applying blockchain technology and smart contracts to modernize digital ticketing. The system uses Solidity contracts to manage ticket booking, ownership records, and verification. This demonstrates how emerging technologies can improve the reliability and transparency of digital infrastructure.

Overall, the proposed system demonstrates how blockchain-based digital ticketing can contribute to innovation, reliable digital infrastructure, and greater transparency. Its impact depends on secure implementation, appropriate access controls, and reliable ticket verification procedures.

10. REFERENCES

1. S. Rafati Niya, S. Bachmann, C. Brasser, M. Bucher, N. Spielmann, and B. Stiller, “DeTi: A Decentralized Ticketing Management Platform,” Journal of Network and Systems Management, vol. 30, article 62, 2022. https://doi.org/10.1007/s10922-022-09675-3.

2. Ethereum Foundation, “Ethereum Developer Documentation.” https://ethereum.org/developers/docs/.

3. Solidity Team, “Solidity Documentation.” https://docs.soliditylang.org/

4. Ayush Gupta, “Blockchain-Based Event Ticketing System,” GitHub repository. https://github.com/AyuShGuptA1302/Blockchain-Based-Event-Ticketing-System

5. Web3.js Contributors, “Web3.js Documentation.” https://docs.web3js.org/.

6. Truffle Suite, “Truffle Suite Documentation.” https://archive.trufflesuite.com/docs/

7. Truffle Suite, “Ganache Documentation.” https://archive.trufflesuite.com/ganache/

8. React Team, “React Documentation.” https://react.dev/

9. Ethereum Foundation, “Smart Contracts.” Ethereum.org. https://ethereum.org/en/developers/docs/smart-contracts/

10. Ethereum Foundation, “Accounts.” Ethereum.org. https://ethereum.org/en/developers/docs/accounts/

11. W. Mougayar, The Business Blockchain: Promise, Practice, and Application of the Next Internet Technology. Wiley, 2016.

12. A. M. Antonopoulos and G. Wood, Mastering Ethereum: Building Smart Contracts and DApps, 1st ed. O’Reilly Media, 2018. https://github.com/ethereumbook/ethereumbook

11. APPENDIX A

https://docs.google.com/document/d/1oM1HZflHr7kVbFfi2aIuIt5HGbQx7i_Y/edit?usp=sharing&ouid=100491147756649966059&rtpof=true&sd=true